CPP NOTES – DAY 22

What is a Segmentation Fault?

A segmentation fault occurs when a program tries to access memory that it is not allowed to. This can happen for several reasons, such as:

- Accessing invalid memory addresses (e.g., dereferencing a NULL pointer or accessing out-of-bounds memory).
- Reading or writing memory that has already been freed (e.g., using a dangling pointer).
- Buffer overflows (writing beyond the allocated space for an array or buffer).
- Stack overflows (too much recursion or too much local variable allocation).

When the operating system detects that the program is trying to access memory incorrectly, it terminates the program to prevent further damage or corruption of the system. This results in a segmentation fault.

What is a Core Dump?

A core dump (also called a core file) is a file that is generated when a program crashes due to a segmentation fault or other fatal error. It contains a snapshot of the program's memory at the time of the crash, including:

- The program's call stack.
- The contents of the program's memory (such as variables, heap, stack, etc.).
- The register values of the CPU at the time of the crash.
- The location in the code where the crash occurred.

Core dumps are extremely useful for debugging. Developers can analyze the core dump to figure out why the program crashed and what went wrong. Typically, this can be done using a debugger like gdb (GNU Debugger) to inspect the contents of the memory at the time of the crash.

When does a Core Dump Occur?

A core dump is created when:

1. Segmentation Fault: The program accesses restricted memory or tries to read/write invalid memory.
2. Abort Signal: The program explicitly calls abort(), such as after detecting an error.
3. Other Fatal Errors: For example, an illegal instruction or division by zero can also cause core dumps, depending on the operating system and settings.

Dynamic memory aloc and Pointers

Dynamic memory allocation **returns a pointer** to the block of memory it allocates. You **must use pointers** to interact with dynamically allocated memory.

FYI

- *Dynamically allocated blocks are said to be unnamed addresses.*
- *RAM cannot have negative addresses*
- *In RAM there will be dynamic memory and static memory with a large gap.*
- *Stack memory is known before to loader*
- *Dynamic memory is meant for the variable we don't, whose capacity is allocated at runtime.*
- *Null,malloc,calloc and realloc is defined in stdlib*
- *A pointer cannot store a value, can only store address of a variable(special value).*